

Lab 1: Cluster Bootstrap & Lifecycle (65 min)

Learning Objectives

By the end of this lab you will be able to:

- Start a real 3-node CockroachDB cluster and confirm it is healthy
- Explain how the nodes found each other: `--join`, `--advertise-addr`, and `cockroach init`
- Inspect node liveness, range distribution, and leaseholders from SQL
- Kill a node and watch Raft keep serving reads and writes
- Restart a node and watch it rejoin with its data intact
- Drive a graceful **decommission** — and discover why it needs `replication factor + 1` nodes
- Connect three ways: the built-in shell, `psql`, and a Python driver
- Diagnose the common "my cluster won't start" failures

Prerequisites

- **Docker Desktop** (or Docker Engine) running, with at least 4 GB available to it
- A modern browser for the DB Console
- Nothing else. There is no `cockroach` binary to install — every node, and the SQL shell itself, runs in a container.

Windows, macOS, Linux. The commands below are identical everywhere. This lab writes `scripts/crdb` to mean "the wrapper for your shell":

Shell	Use
macOS, Linux, WSL	<code>scripts/crdb.sh</code>
Windows <code>cmd</code> / PowerShell	<code>scripts\crdb.bat</code>

On Windows, **WSL is the recommended path** — it is Linux, so every command in the course works verbatim.

Setup

From the repository root:

```
scripts/crdb up
```

That starts three nodes (`crdb1`, `crdb2`, `crdb3`), runs `cockroach init` once, and prints the node list. It takes about 20 seconds the first time while the image downloads.

Open a SQL shell whenever the lab asks for one:

```
scripts/crdb sql
```

And the DB Console at <http://localhost:8080>.

Tasks

Part A: A Healthy Cluster (15 min)

1. **Confirm cluster size and identity** — in a SQL shell (`scripts/crdb sql`):

```
SHOW CLUSTER SETTING version;

SELECT node_id, address, locality, is_live
FROM crdb_internal.gossip_nodes
ORDER BY node_id;

SELECT count(*) AS total_ranges FROM crdb_internal.ranges_no_leases;
```

You should see three live nodes and ~30–50 system ranges (no user data yet).

2. **Create a small dataset:**

```
CREATE DATABASE lab1;
USE lab1;

CREATE TABLE notes (
  id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  body  STRING NOT NULL,
  made  TIMESTAMPTZ DEFAULT now()
);

INSERT INTO notes (body)
SELECT 'note ' || generate_series(1, 1000)::STRING;

SELECT count(*) FROM notes;
```

Expected count: 1000.

3. **Find the leaseholder:**

```
SHOW RANGES FROM TABLE notes WITH DETAILS;
```

Note the `lease_holder` column — typically a single range for 1000 rows. Every read and write for that range goes through exactly one node; the other two hold replicas.

4. Look at the DB Console (<http://localhost:8080>):

- **Overview** — 3 live nodes, all green
- **Databases** → `lab1` → `notes` — row count, size
- **Hot Ranges** — quiet right now; you'll come back to this in Lab 3

Where is the data? Each node writes to a Docker named volume (`crdb-labs_crdb1-data` and friends), so a container can stop and start without losing anything. `scripts/crdb down` removes the volumes as well — that is the "start over" button.

Part B: How This Cluster Was Formed (10 min)

Read `docker/labs.yml` while answering these. Everything a production deployment does, this file does in miniature.

1. Find the three flags that let the nodes find each other:

```
--join=crdb1,crdb2,crdb3    every node is told the same set of peers
--advertise-addr=crdbN      the address a node tells the others to reach it on
--insecure                  no TLS (Lab 12 does it properly)
```

`--join` is not "the leader". There is no leader to point at — each node just needs enough addresses to find the gossip network. Listing all three is the normal pattern.

2. Find the `init` service. Starting nodes is not the same as having a cluster: until something runs `cockroach init`, the nodes sit waiting, logging that they are not initialised. `init` runs exactly once, against one node, and the cluster comes alive.

```
scripts/crdb run node status --insecure
```

3. Prove `init` is one-time. Run it again by hand:

```
docker compose -f docker/labs.yml exec crdb1 ./cockroach init --insecure
```

It refuses — the cluster is already initialised. This is why the `init` service in the compose file exits 0 either way.

4. Where the DB Console ports come from: each node publishes `8080` inside the container; compose maps them to `8080`, `8081`, `8082` on your machine. The SQL port `26257` maps to `26257`, `26258`, `26259`. Check with:

```
scripts/crdb ps
```

Part C: Kill a Follower, Watch Recovery (10 min)

1. Note which node holds the lease for `notes` :

```
SELECT range_id, lease_holder FROM [SHOW RANGES FROM TABLE lab1.notes WITH DETAILS];
```

2. Stop a node that is *not* the leaseholder (say the lease is on node 1, stop node 3):

```
scripts/crdb stop 3
```

3. Reads and writes keep working — the remaining two nodes are a quorum of three:

```
SELECT count(*) FROM lab1.notes;  
INSERT INTO lab1.notes (body) VALUES ('written while a node was down');  
SELECT count(*) FROM lab1.notes;
```

4. Watch the cluster notice:

```
SELECT node_id, is_live FROM crdb_internal.gossip_nodes ORDER BY node_id;
```

The stopped node flips to `false` within a few seconds. In the DB Console the node turns red, and **Metrics** → **Replication** shows under-replicated ranges.

5. Bring it back:

```
scripts/crdb start 3
```

```
SELECT node_id, is_live FROM crdb_internal.gossip_nodes ORDER BY node_id;
```

It rejoins with the **same node ID** and its existing data, then catches up on what it missed. That is why the volume matters: without it the node would come back empty and every range would have to be re-replicated to it from scratch.

6. Wait for full replication before moving on:

```
SELECT count(*) AS under_replicated  
FROM crdb_internal.ranges_no_leases  
WHERE array_length(replicas, 1) < 3;
```

Keep running it until it reports 0.

Part D: Kill the Leaseholder (5 min)

1. Find the current leaseholder for `notes` and stop *that* node:

```
SELECT range_id, lease_holder FROM [SHOW RANGES FROM TABLE lab1.notes WITH DETAILS];
```

```
scripts/crdb stop <that-node-number>
```

2. Query again immediately:

```
SELECT count(*) FROM lab1.notes;
```

The first query may pause for a moment while a surviving replica takes over the lease, then succeeds.

No data was lost and no human intervened. That pause — a few hundred milliseconds to a couple of seconds — is what "multi-active availability" costs you on a node failure.

3. Restart it:

```
scripts/crdb start <that-node-number>
```

Part E: Decommission Needs Spare Capacity (15 min)

Decommissioning is how you *shrink* a cluster on purpose — different from a kill, because the cluster moves every replica off the node before letting it leave.

1. Try to decommission node 3 — and watch it be refused:

```
scripts/crdb run node decommission 3 --insecure
```

```
ranges blocking decommission detected
n3 has 67 replicas blocked with error: "0 of 2 live stores are able to take a new
replica for the range (2 already have a voter, 0 already have a non-voter);
likely not enough nodes in cluster"
ERROR: Cannot decommission nodes.
```

This is the lesson. With 3 nodes and a replication factor of 3, every range already has a replica on all three — there is nowhere for node 3's replicas to go, so the cluster refuses rather than dropping below its replication target.

You need replication factor + 1 nodes to shrink. This is the most common surprise when someone tries to scale a 3-node cluster down, and it is why Lab 16 insists on decommissioning *before* reducing the Kubernetes replica count — the same constraint, one layer up.

2. Add a fourth node so the replicas have somewhere to go:

```
scripts/crdb add-node
scripts/crdb run node status --insecure
```

3. Decommission node 3 again — now it works:

```
scripts/crdb run node decommission 3 --insecure
```

The command blocks until every replica on node 3 has a home elsewhere. On a cluster this small that is seconds; on a production node it can be many minutes.

4. Confirm:

```
SELECT node_id, membership FROM crdb_internal.kv_node_liveness ORDER BY node_id;
```

Node 3 reports `decommissioned`; the others are `active`. Reads and writes worked throughout.

5. What recommission can and cannot do:

```
scripts/crdb run node recommission 3 --insecure
```

Recommission reverses a decommission that is still *in progress*. Once a node reaches `decommissioned` it is permanently out; bringing that capacity back means starting a new node, which joins with a new node ID.

6. Reset for the next part:

```
scripts/crdb reset
```

Part F: Three Ways to Connect (10 min)

CockroachDB speaks the PostgreSQL wire protocol, and the lab cluster publishes port `26257` on your machine — so any PostgreSQL client works, containerised or not.

1. The built-in shell (what you have been using):

```
scripts/crdb sql -e "SELECT count(*) FROM lab1.notes;"
```

2. `psql` — if you have it locally:

```
psql "postgresql://root@localhost:26257/lab1?sslmode=disable" -c "SELECT count(*) FROM notes;"
```

No local `psql` ? Run one in a container instead:

```
docker run --rm --network crdb-labs_default postgres:16 \
  psql "postgresql://root@crdb1:26257/lab1?sslmode=disable" -c "SELECT count(*) FROM notes;"
```

3. Python — again, no local install needed:

```
docker run --rm --network crdb-labs_default -e PGPASSWORD= python:3.12-slim bash -c \
  "pip install -q psycopg2-binary && python -c \"
import psycopg2
c = psycopg2.connect('postgresql://root@crdb1:26257/lab1?sslmode=disable')
cur = c.cursor(); cur.execute('SELECT count(*) FROM notes'); print('rows:', cur.fetchone()
[0])\""
```

Two addresses, one cluster. From your machine it is `localhost:26257` (the published port). From another container on the same Docker network it is `crdb1:26257`. Mixing them up is the most common connection error in the rest of this course.

Optional — If Time Allows

These parts are not required to complete the lab; they extend it by about 10 minutes. Do them if you finish early, or after class — the cluster and data from the core parts carry over.

Part G: Troubleshooting a Cluster That Won't Start (10 min)

Break it deliberately — these are the failures you will actually meet.

1. **Port already in use.** Something else on your machine holds 26257:

```
docker run --rm -p 26257:26257 alpine sleep 30 &
scripts/crdb reset
```

You get `port is already allocated`. Fix: stop the other process, or change the published port in `docker/labs.yml`.

```
docker ps | grep alpine    # find and stop it
```

2. **Nodes start but the cluster never comes up.** Simulate it by starting a node with a `--join` list nobody else shares:

```
docker run --rm --network crdb-labs_default cockroachdb/cockroach:v23.2.5 \
  start --insecure --advertise-addr=lonely --join=nosuchnode:26257 --http-
  addr=0.0.0.0:8080
```

It logs that it cannot reach its join targets and waits forever. A node with a wrong `--join` never errors out — it *hangs*, which is why this one is hard to spot in production. Ctrl+C to stop it.

3. **Reading the logs:**

```
scripts/crdb logs 1
```

Look for `node starting`, `initialized`, and any `join` warnings. Ctrl+C to stop tailing.

4. When a node is unhealthy, check it directly:

```
scripts/crdb ps  
docker compose -f docker/labs.yml exec crdb1 ./cockroach node status --insecure --all
```

Cleanup

```
scripts/crdb down
```

That stops all four nodes and deletes their volumes. The next lab starts from a clean `scripts/crdb up`.

To keep the data and just stop the containers, use `docker compose -f docker/labs.yml stop`.

Lab 1 Deliverables

□ **A running 3-node cluster** in Docker, with 1000 rows and a known leaseholder □ **Cluster formation understood** — `--join`, `--advertise-addr`, and one-time `cockroach init` □ **Follower failure** survived; reads and writes continued □ **Leaseholder failure** survived, with the lease-transfer pause observed □ **Node restart** rejoined with the same node ID and its data □ **Decommission** refused at 3 nodes, then completed at 4 — with the reason understood □ **Three client paths** — built-in shell, `psql`, Python driver □ **Two failure modes** reproduced: port conflict and a bad `--join`

Challenge Exercises

1. **Kill two of three nodes.** What happens to reads? To writes? Explain it in terms of quorum, then bring them back and confirm nothing was lost.
2. **Change the replication factor to 5** on a 3-node cluster
(`ALTER RANGE default CONFIGURE ZONE USING num_replicas = 5;`) and watch `SHOW RANGES`. What does the cluster do, and what does the DB Console say about it?
3. **Add a `--locality` flag** to each node in `docker/labs.yml` (`--locality=region=us-east1,zone=a` and so on), `scripts/crdb reset`, and confirm `crdb_internal.gossip_nodes` shows it. This is the setup Lab 7 builds on.
4. **Make node 2 the leaseholder for `notes` on purpose.** Find the syntax (`ALTER RANGE ... RELOCATE LEASE`), and verify with `SHOW RANGES`.

Reference

Command	Purpose
<code>scripts/crdb up</code>	Start and initialise the 3-node cluster
<code>scripts/crdb sql</code>	SQL shell on node 1
<code>scripts/crdb sql -e "..."</code>	Run one statement non-interactively
<code>scripts/crdb sql-on 2</code>	SQL shell on a specific node
<code>scripts/crdb run <args></code>	Any <code>cockroach</code> subcommand on node 1
<code>scripts/crdb stop N / start N</code>	Simulate a node failure and recovery
<code>scripts/crdb add-node</code>	Start a 4th node
<code>scripts/crdb status / ps / logs N</code>	Node status, container status, logs
<code>scripts/crdb down</code>	Delete the cluster and its data
<code>scripts/crdb reset</code>	down then up
<code>crdb_internal.gossip_nodes</code>	Node liveness and locality
<code>SHOW RANGES FROM TABLE t WITH DETAILS</code>	Ranges, sizes, leaseholders
<code>crdb_internal.kv_node_liveness</code>	Membership: active / decommissioning / decommissioned